

# Subsetting

|                    |                                    |
|--------------------|------------------------------------|
| Document #         | D3716R0                            |
| Date               | 2025-05-19                         |
| Targeted subgroups | EWG, SG23                          |
| Ship vehicle       | C++29                              |
| Reply-to           | Peter Bindels <dascandy@gmail.com> |

What does "-Wall" in "g++ -Wall test.cpp -o test" do? -- It's short for "warn all"; it turns on (almost) all the warnings that g++ can tell you about. Typically a good idea, especially if you're a beginner, because understanding and fixing those warnings can help you fix lots of different kinds of problems in your code.

## § 1 Abstract

We propose to have a standard facility in C++ to define a subset of the language, and to enforce a subset of the language in a given environment.

## § 2 Prior art

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1881r1.html>

- An epoch could reduce the number of possibilities and the complexity of the language by forbidding a subset of the existing approaches
- The author of this paper has delivered C++ training to hundreds of people of different skill levels, and strongly believes that the complexity of topics such as variable initialization could be eradicated by using a mechanism like epochs. After explaining how to enable the latest epoch to students, the training could focus on a safe and logical subset of the latest standard that does not provide needlessly varied and complicated choices. Furthermore, students attempting to use unsafe constructs that they learned from C or poor C++ training material would be stopped by the compiler before introducing undefined behavior into their code.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3081r1.pdf>

- Define standard enforced “profiles” that a conforming C++ implementation must enforce when enabled, notably bounds, type, and lifetime. This is in addition to any user-defined profiles.

- Each profile consists of rules. Each rule must be deterministically decidable at compile time (even if it results in injecting a check enforced at run time) and must be sufficiently efficient to implement in-the-box in the C++ compiler without unacceptable impact on compile time.
- Rules are portable and enforced in the C++ implementation, not in a separate tool such as a static analyzer.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3390r0.html>

- A superset of C++ with a safe subset. Undefined behavior is prohibited from originating in the safe subset.
- The safe and unsafe parts of the language are clearly delineated. Users must explicitly leave the safe context to write unsafe operations.
- The safe subset must remain useful. If we get rid of a crucial unsafe technology, like unions and pointers, we should supply a safe alternative, like choice types and borrows. A safe toolchain is not useful if it's so inexpressive that you can't get your work done.

<https://wg21.link/p2759>

- Profiles package up several features to make it visible for a code region. Profiles do not limit code in such a way that it reduces the language expressivity like subsets do. We do recognize some domains can deal with subsets and are thus not opposed to a profile-specific subset. However, it is our opinion that subsetting is not a suitable solution for a general purpose language.

Reddit [https://www.reddit.com/r/cpp/comments/ee3a48/subset\\_of\\_c/](https://www.reddit.com/r/cpp/comments/ee3a48/subset_of_c/)

- ... is it possible to have a subset of modern C++. ... With such a massive focus on modern C++ and teaching people about all the RAII techniques, smart pointers, containers, STL, algorithms and so much more, is it possible to just have a subset of C++, which enforces these best practices by default and let people study only the new/modern aspects of C++ leaving behind the legacy versions?
- This idea comes up, if you ask me, surprisingly often.
- C++ can take leaf out of Rust's notebook. The language allows you to mark code as unsafe code which lets you do some C style coding. Similarly this subset of C++ can allow developers to mark code as legacy or some other keyword and proceed with it.
- C/C++ is what actually backwards. Safe code should be the default to make writing safe programs effortless. Full freedom to do anything is what caused tons of these memory related vulnerabilities that plague any C/C++ software.

StackOverflow <https://stackoverflow.com/questions/3073642/official-c-language-subsets>

- I've been restricting myself to a very C-like subset of C++ features; namely, no classes/inheritance except complex and STL, templates only used for find/replace kinds of substitutions, and a few other things I can't put in words off the top of my head. I am wondering if there are any official or well-documented subsets of the C++ language that I could look at for reference (as well as rationale) when I go about picking and choosing which features to use.
- Google publishes its internal C++ style guide, which is often referred to as such a subset: <https://google.github.io/styleguide/cppguide.html>
- The SEI CERT C++ Coding Standard gives a list of rules for writing safe, reliable, and secure systems in C++14. This is not a subset of C++ per se, but as a coding standard like the other answers is a subset in effect by avoiding unsafe, undefined, or easily-misused features (including some common to C).
- How close is existing C/C++ code to a safe subset?  
<https://www.mdpi.com/2624-800X/4/1/1>
- Using a safe subset of C++ is a promising direction for increasing the safety of the programming language while maintaining its performance and productivity. In this paper, we examine how close existing C/C++ code is to conforming to a safe subset of C++. We examine the rules presented in existing safe C/C++ standards and safe C/C++ subsets.
- We find that raw pointers, unsafe casts, and unsafe library functions are used in both C/C++ code at large and in modern C++ applications. In general, C/C++ code at large does not differ much from modern C++ code, and continued work will be required to transition from existing C/C++ code to a safe subset of C++.

## § 3 Existing subsetting of C++

- In hard-embedded setups, dynamic allocations are not allowed
- IAR long shipped a mode called "Embedded C++"
- GCC / Clang ship a "-fno-rtti -fno-exceptions" mode, that disable RTTI and exceptions
- Many people want to use the "without-C" subset of C++
- Library authors want to use the "C++17-compatible" subset, typically enabled with `-std=c++17`
- MISRA and AutoSAR users want to use the compliant subset
- C added Annex K, subsetting out undesired functions
- Microsoft Visual C++ added the C4996 warning, subsetting out undesired functions
- the Clang/GCC `-Wall -Werror`
- the Clang/GCC `-Wall -Wextra -Werror`
- the Clang/GCC `-Wall -Wextra -Wpedantic -Werror`

- In "Modern C++", we want to avoid raw "new" and "delete" statements in user code

## § 4 Design principles

- Code either compiles and works identically to what it does without the subset, or it does not compile. There are no other possible outcomes.
- Subsets always combine orthogonally. There are no interactions between subsets, no changed behavior.
- The compiler and linker do not get any special knowledge or permissions from the existence of a subset in a part of the program.
- Subset specification is done by many different unrelated standard bodies and code owners.
- Suppressing a given subsetting rule must be portable without requiring arbitrarily-large suppression lists.

## § 5 Why is subsetting a thing we can and want to do?

- It does not change the meaning of any code

The only thing it allows is removing a construct, function, type or keyword from use. The only change a user can see to their program is that it is now ill-formed, with a specific indication where the given subset is violated.

- The majority of code is not trying to do most of the things the language can do

Most people have used an axe and a gun at some point in their life, but don't use axes or guns often. Similarly, most C++ code ends up relying on pointer arithmetic, but does not try to do any pointer arithmetic by itself. Rules in subsets can be suppressed, allowing for a nearly-always rule to still be enabled.

- Subsets combine orthogonally

Subsets define specific actions that are disallowed. The sum of two subsets is the sum of their disallowed actions. If any subset disallows suppressing a given rule, the sum subset disallows suppressing that rule.

- It is common for people to subset the language, and dozens of subsets are in common use

Building with warnings-as-errors for a warning set, subsetting out the warning-causing constructs. Building in `-std=c++17` mode, subsetting out all C++20+

constructs. Building with `-fno-exceptions -fno-rtti`, disabling exceptions and RTTI.

- It is one part of the mosaic of changes needed to create a safe future C++ language

## § 6 How to subset

Define a subset by doing one or more of the following

- Disallow language keyword entirely
- Disallow use of a specific type
- Disallow specific function usage (ie, mark function as effectively `=delete` despite being defined properly)
- Disallow enumerated set of specific language actions (array decay, variadic function use, pointer arithmetic, ...)
- Including a full named subset as part of this subset

Each subset indicates whether the rules it disallows are suppressible. The set of subsets should be open-ended, so that other organizations (SEI, MISRA, AutoSAR, LLVM, Microsoft etc) can define subsets.

They are allowed to use the knowledge gained from a subset while compiling a TU, and can use the knowledge of the subset in linking if they can be certain that all TUs were compiled with that subset - all under the existing as-if rule.

## § 7 Evolving subsets over time

A subset should have a semantic meaning, a user-understandable goal of the profile. The semantic meaning is what determines which rules should be included in a subset. Most subsets are defined once and do not naturally accumulate more rules over time, because they are restricting the existing language to remove particular features that are not being newly added. Some subsets however, particularly those oriented around new language feature restriction, naturally accumulate new rules over time as new features are added to the language. In a different way, companies tend to maintain their own subset of the language roughly corresponding to "all the warnings we've been able to fix around our software", where a company tries to expand the subset over time, making sure that the old subset is strictly not violated, while attempting to add new rules that can be fixed in software over time, preventing those same issues from showing up in future code.

The first of these will naturally accumulate more rules over time, but retains the same meaning. These subsets can be evolved in place.

The second of these will also accumulate more rules over time, as the company using it will increase the set of rules they are enforcing over time. In this case though, the logical meaning of the subset is different; the best description for the logical subsets is likely "The set of warnings we enforce in 2024 and on", and similarly "The set of warnings we enforce in 2026 and on". The latter naturally includes the former, and expands on it, and as such the subsets should themselves be written as composed subsets.

If a subset is found to contain a rule that should be omitted, it is possible to remove the rule, as it only relaxes the subset allowing more of the full language to be used.

## § 8 Suppressing a rule or a subset?

Suppressing rules is more verbose; a statement can have multiple rules disallowing it.

Suppressing subsets requires a closed set of subset definitions, so that the suppression can target it. We see a major desire in many places to define subsets to correspond to restrictions that various groups want. At the moment we could think of compiler-designed subsets (removing anything from after C++17, removing all -Wall-triggering constructs), standard-body subsets (removing all constructs that violate MISRA rules), standard C++ subsets (removing all constructs that are considered obsolete in C++29), regulatory subsets (removing specific constructs considered to be unacceptable) and code-owner subsets (removing the constructs that the owner has removed and wants to make sure the code base is devoid of).

We propose to add suppressions on *\*rules\**. This makes it so that the subset definitions can be varied by users, allowing (for example) MISRA and AutoSAR to update their definitions, and for things disallowed by multiple subsets to only need a single suppression. It also has the subtle effect of making code that breaks various subsetted properties to need multiple suppressions, making "worse" code "smell worse". In addition, it has the benefit that if a rule is disallowed by multiple subsets in use that the suppression works across all of them.

## § 9 Example use

No exceptions subset: Disallow use of catch keyword.

No RTTI subset: Disallow use of catching non-final types, disallow use of the `dynamic_cast` keyword, disallow typeid run on a non-type argument.

Annex K subset: Disallow use of all functions mentioned in Annex K.

Type-safety subset: Disallow use of `reinterpret_cast`, `const_cast` etc. as described in paragraph 4 of [p3081](#).

C++17 subset: Disallow use of the C++20 subset, plus all changes between C++17 and C++20 (not listed).

## § 10 Wording

To be added if the paper is marked as desirable by EWG / SG23